

Factory Safety Incident Tracker

Week 3 Deliverable — Requirements, Architecture and Technology Setup

1. SRS Summary

Functional Requirements

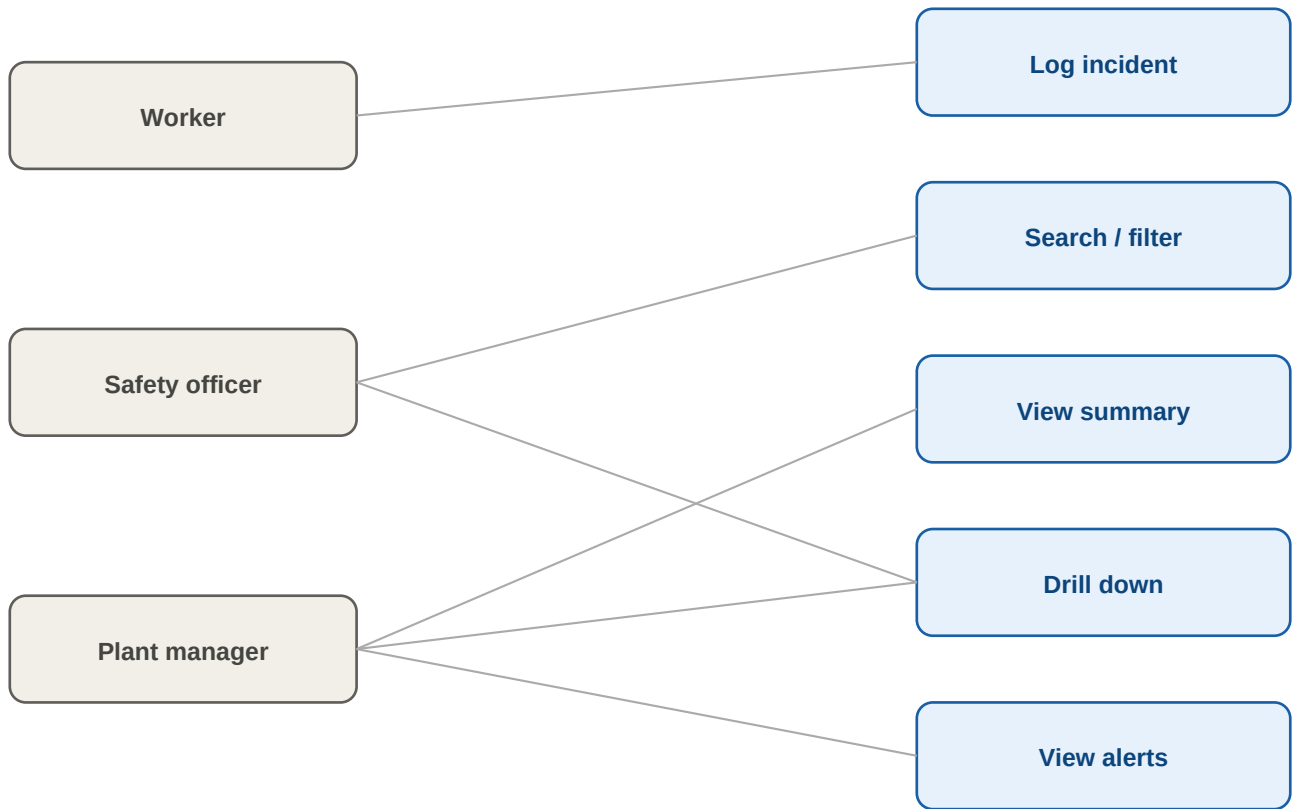
- FR1: The system shall allow a worker to submit an incident with title, description, severity, location, and reporter.
- FR2: The system shall allow a safety officer to search/filter incidents by date, status, severity, and location.
- FR3: The system shall display summary counts of open, closed, and high-severity incidents.
- FR4: The system shall allow drilling from a summary indicator into a filtered list and then a detail view.
- FR5: The system shall surface an alert/exception view of unresolved high-severity incidents.

Non-Functional Requirements

- **Performance:** dashboard and summary views load in under 3 seconds for up to 500 incidents.
- **Usability:** incident entry completable in under 2 minutes, no training required.
- **Portability:** deployable on Tomcat or Nginx; later containerized with Docker.
- **Reliability:** zero data loss — every submitted incident is persisted.
- **Maintainability:** layered architecture (controller/service/data) to keep components independently testable.

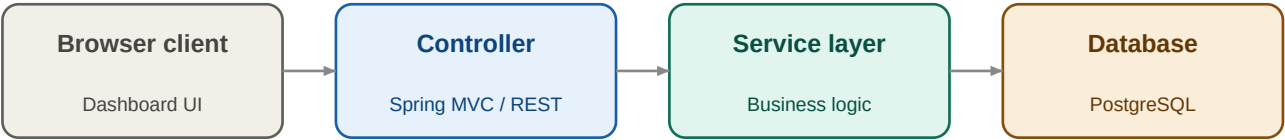
2. Use-Case Diagram

Three actors interact with the system's five MVP use cases, as shown below.



3. Architecture Diagram

The MVP uses a simple layered architecture: a browser client calls a controller/web layer, which delegates to a service layer, which reads and writes the database. The controller and service layer are packaged as a WAR and deployed on Tomcat.



Packaged as a WAR, deployed on Tomcat

4. Data Model

Entity: Incident

Field	Type	Notes
id	UUID / bigint	Primary key
title	varchar(150)	Required
description	text	Optional
severity	enum (Low, Medium, High)	Required
status	enum (Open, In Progress, Closed)	Default: Open
location	varchar(100)	Required
reported_by	varchar(100)	Worker name/ID
reported_at	timestamp	Auto-set on creation
resolved_at	timestamp	Nullable, set on close

Entity: User (minimal, no auth in MVP, used only for attribution): id, name, role.

5. REST API Endpoint List

Method	Endpoint	Purpose
POST	/incidents	Create a new incident
GET	/incidents	List/search incidents (supports ?status=&severity=&location=&from=&to=)
GET	/incidents/{id}	Get single incident detail

Method	Endpoint	Purpose
PUT	/incidents/{id}/status	Update status (Open to In Progress to Closed)
GET	/incidents/summary	Return counts: open, closed, high-severity
GET	/incidents/alerts	Return unresolved high-severity/overdue incidents

6. Working Local Setup

- 1 Install JDK 17, Maven, and PostgreSQL (or use H2 for a zero-install local DB).
- 2 Bootstrap the project at start.spring.io with dependencies: Spring Web, Spring Data JPA, PostgreSQL Driver (or H2), Lombok.
- 3 Configure `application.properties` with your DB connection.
- 4 Run `mvn spring-boot:run` and confirm the app starts on `localhost:8080`.
- 5 Hit POST `/incidents` with a test payload (via curl/Postman) and confirm a row appears in the DB.
- 6 Screenshot this working end-to-end call as your Week 3 setup evidence.